

Manual de Estudio y Defensa Técnica

Modulo de Mensajes de Chat — Plataforma SaaS de CRM + WhatsApp
Arquitectura, frameworks, principios de programacion y analisis de sistemas

Entregable	URL
Frontend (Vercel)	https://chat-frontend-ochre-psi.vercel.app/chats/1
API (Cloudflare Workers)	https://chat-backend.gabrielmarves.workers.dev
Repositorio	https://github.com/JGMarves17/prueba-tecnica-chat

Como usar este manual

Cada capitulo responde a una familia de preguntas que un evaluador tecnico suele hacer. Los apartados marcados como **Defensa** contienen la respuesta corta que conviene tener preparada, y los marcados como **Punto debil** son las criticas legitimas que se pueden recibir: reconocerlas con criterio puntua mas que fingir que no existen.

Stack: Hono · TypeScript · Drizzle ORM · Neon Postgres · Cloudflare Workers · Next.js 14 App Router · TanStack Query v5 · Tailwind CSS · Vitest

Indice

1. Resumen ejecutivo del sistema

Que resuelve · Alcance funcional · Metricas del entregable

2. El stack tecnologico, decision por decision

Hono · Cloudflare Workers · Neon · Drizzle · Next.js · TanStack Query · Tailwind · Zod · Vitest

3. Arquitectura del sistema

Vista de despliegue · Arquitectura en capas · Ciclo de vida de una peticion · Edge y arranque en frio

4. Analisis y diseno de sistemas

Requisitos · Casos de uso · Modelo entidad-relacion · Diccionario de datos · Flujos

5. Principios de programacion aplicados

SoC · SRP · DRY · Fail fast · Fuente unica de verdad · No confiar en el cliente · Idempotencia · Tipado como contrato

6. Patrones de diseno identificables

Middleware chain · Inyeccion de dependencias · DTO y contrato · Optimistic UI · Contenedor/presentacion · Guard clause

7. El contrato de la API

Formato de respuesta · Endpoints · Codigos de estado · Validaciones

8. Estrategia de pruebas

Piramide de testing · Los 15 tests · Que NO cubren · Verificacion manual

9. Seguridad

CORS · Gestion de secretos · Inyeccion SQL · XSS · Superficie de ataque

10. Despliegue y operacion

Topologia · Variables de entorno · Migraciones · Seed idempotente

11. Defensa: preguntas y respuestas

30 preguntas de evaluador con respuesta preparada

12. Limitaciones conocidas y hoja de ruta

Deuda tecnica reconocida · Que haria distinto a escala

13. Glosario

Terminos que conviene saber definir

1. Resumen ejecutivo del sistema

El sistema implementa el **módulo de mensajes** de una plataforma SaaS que combina un chatbot de WhatsApp con un CRM. En el modelo de negocio, cada cliente de la plataforma es una **empresa**; cada empresa gestiona muchos **chats** (conversaciones con sus clientes finales); y cada chat acumula muchos **mensajes** que van y vienen. El equipo de atención necesita ver el historial de una conversación y responder desde el panel.

Se trata por tanto de un sistema **multi-tenant**: una sola instancia de software sirve a múltiples empresas, y la tabla empresas es la raíz de esa separación lógica.

1.1 Alcance funcional entregado

Capacidad	Implementación
Listar el historial de un chat	GET /chats/:chatId/mensajes, orden cronológico ascendente, con paginación
Responder desde el panel	POST /chats/:chatId/mensajes; el servidor fija dirección = saliente
Eliminar un mensaje	DELETE /mensajes/:id, con confirmación en la interfaz
Contexto de la conversación	GET /chats/:chatId devuelve nombre del contacto y teléfono
Diagnóstico operativo	GET /health, deliberadamente sin dependencia de base de datos

1.2 Métricas del entregable

Indicador	Valor
Endpoints HTTP	5 (incluido el health check)
Tablas en base de datos	3 (empresas, chats, mensajes)
Tests unitarios	15, todos en verde
Tamaño del bundle del Worker	563 KiB sin comprimir / 124 KiB gzip
Arranque en frío del Worker	13 ms medidos en el despliegue
Rutas del frontend	3 (/ , /chats/[id], /_not-found)
JavaScript de la página de chat	108 KB First Load JS
Commits	22, con mensajes descriptivos
Coste de infraestructura	0 EUR: los tres servicios en plan gratuito

Defensa: por que este alcance

El enunciado avisa de que el objetivo no es completar el 100%, sino demostrar criterio. La decisión fue **cerrar por completo el flujo principal** (listar, enviar, borrar) con validación, manejo de errores, pruebas y despliegue real, en lugar de dejar a medias un abanico mas amplio de funciones. Un módulo terminado y desplegado se puede auditar; media docena de funciones incompletas, no.

2. El stack tecnologico, decision por decision

Cada pieza del stack venia impuesta por el enunciado. Aun asi, conviene saber **que hace cada una, que problema resuelve y cual era la alternativa**: es la pregunta mas probable de toda la defensa.

2.1 Backend

Pieza	Que es	Por que encaja aqui	Alternativa
Hono	Framework web ultraligero basado en la Web Fetch API estandar	Express depende de las APIs de Node y no funciona en un runtime edge. Hono habla Request/Response nativos, que es justo lo que expone Workers	Express, Fastify, itty-router
Cloudflare Workers	Runtime serverless en el edge, con aislados de V8 en lugar de contenedores	Sin arranque en frio apreciable (13 ms medidos) y distribucion global. No hay servidor que mantener ni escalar manualmente	AWS Lambda, Vercel Functions, Deno Deploy
Neon	Postgres serverless que separa computo y almacenamiento	Postgres real, no un dialecto. Su driver HTTP funciona en el edge, donde no existen los sockets TCP que usa el protocolo nativo de Postgres	Supabase, PlanetScale, RDS
Drizzle ORM	ORM ligero con tipos inferidos del esquema	Genera SQL predecible, pesa poco (importa en un bundle edge) y deriva los tipos de TypeScript del esquema: una columna mal escrita no compila	Prisma, TypeORM, SQL a pelo
Zod	Validacion de esquemas con inferencia de tipos	Valida en el limite del sistema y de ese mismo esquema se infiere el tipo: una sola definicion sirve para validar y para tipar	Yup, Joi, validacion manual
Vitest	Framework de pruebas compatible con la API de Jest	Arranca en menos de un segundo y entiende TypeScript y ESM sin configuracion extra	Jest, node:test, bun test

2.2 Frontend

Pieza	Que es	Por que encaja aqui	Alternativa
Next.js 14 App Router	Framework de React con enrutado por ficheros y Server Components	El layout es Server Component y solo la parte interactiva viaja al navegador. Ademas el despliegue en Vercel es de primera clase	Vite + React Router, Remix
TanStack Query v5	Gestor de estado de servidor: cache, revalidacion y mutaciones	Un chat es estado de servidor, no de cliente. Query resuelve cache, invalidacion, estados de carga y error y actualizaciones optimistas sin escribir esa maquinaria a mano	SWR, RTK Query, useEffect + useState
Tailwind CSS	CSS por utilidades, compilado bajo demanda	El estilo vive junto al marcado, no se acumula CSS muerto y el tema queda centralizado en un unico fichero de configuracion	CSS Modules, styled-components, shadcn/ui
TypeScript strict	JavaScript con tipos verificados en compilacion	El contrato de la API se declara una vez y el compilador impide consumirlo mal: errores que serian de ejecucion pasan a ser de compilacion	JavaScript con JSDoc

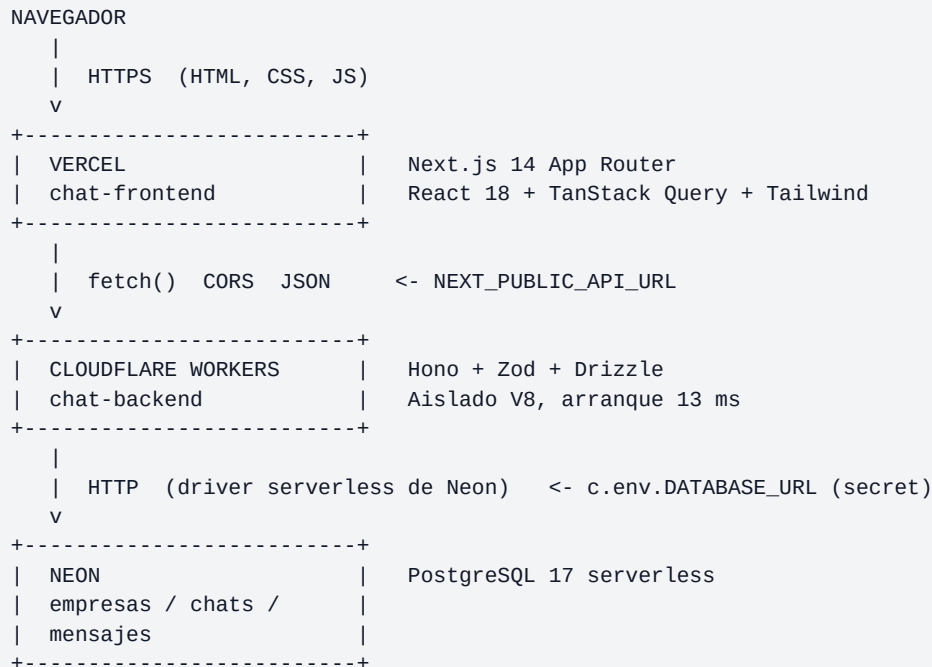
Defensa: la pregunta trampa sobre el ORM

Si preguntan por que Drizzle y no Prisma, que es mas conocido: Prisma arrastra un motor de consultas binario y un cliente generado que **no funcionan en un runtime edge** como Workers sin recurrir a su Data Proxy de pago. Drizzle es una capa fina sobre el driver, se empaqueta sin problema y produce SQL legible. La restriccion del entorno decide la herramienta.

3. Arquitectura del sistema

3.1 Vista de despliegue

Tres servicios independientes, cada uno en su proveedor, comunicados por HTTPS. Ninguno comparte proceso ni sistema de ficheros con los demás.



Por que el navegador llama directamente a Workers y no pasa por Vercel

Seria posible usar Next.js como proxy (Route Handlers o rewrites), pero eso anade un salto de red y una segunda funcion serverless al camino de cada peticion, sin ganar nada: la API ya es publica y no hay secretos que ocultar en ese trayecto. Llamar directo es **mas rapido y mas simple**. El precio es tener que configurar CORS, que es exactamente lo que pide el enunciado.

3.2 Arquitectura en capas del backend

El backend sigue una **arquitectura en capas** clasica: cada capa solo conoce a la inmediatamente inferior, y una peticion las atraviesa en orden.

Capa	Fichero	Responsabilidad	Que NO hace
1. Entrada / infraestructura	src/index.ts	Monta la app, CORS, logging, inyecta la conexion, captura errores no controlados y define el 404	No conoce reglas de negocio ni tablas
2. Validacion	src/utills/validations.ts	Define con Zod la forma valida de params, query y body. Traduce texto de entrada a tipos	No toca la base de datos ni HTTP
3. Rutas / casos de uso	src/routes/chat.ts	Orquesta: valida, comprueba existencia, consulta o escribe, y devuelve la respuesta con el contrato	No abre conexiones ni construye SQL a mano
4. Acceso a datos	src/db/index.ts, src/db/schema.ts	Define el esquema tipado y fabrica la instancia de Drizzle por peticion	No sabe nada de HTTP ni de codigos de estado

3.3 Ciclo de vida de una petición

Recorrido completo de POST /chats/1/mensajes, que es el caso mas rico porque atraviesa todas las capas y todas las validaciones:

1. logger()	Registra metodo y ruta
2. cors()	Evalua el Origin; si no esta permitido, no anade ACA0
3. middleware de BD	c.set("db", getDb(c.env)) <- conexion por peticion
4. zValidator("param")	chatId debe casar con /\d+\$/ -> si no, 400
5. jsonBodyGuard	Intenta parsear el body -> si falla, 400
6. zValidator("json")	contenido: trim + min 1 + max 5000 -> si no, 400
7. handler: existe el chat?	SELECT ... WHERE id = \$1 LIMIT 1 -> si no, 404
8. handler: insertar	INSERT ... direccion = "saliente" <- FIJO en servidor
9. respuesta	201 { status: "success", mensaje: {...} }

Si en cualquier punto salta una excepcion no prevista:

```
app.onError -> 500 { status: "error", message: "Error interno del servidor" }
```

Lectura clave de este flujo

Las comprobaciones estan ordenadas **de la mas barata a la mas cara**. Validar un numero con una expresion regular no cuesta nada; consultar la base de datos, si. Por eso un chatId invalido se rechaza en el paso 4 y nunca llega a tocar Neon. Es fail fast aplicado al rendimiento y al coste.

3.4 Edge computing: que cambia respecto a un servidor tradicional

Aspecto	Servidor tradicional (Node en un VPS)	Cloudflare Workers (este proyecto)
Unidad de ejecucion	Proceso Node de larga vida	Aislado de V8 creado por peticion
Estado en memoria	Persiste entre peticiones	No se puede asumir que persista
Conexion a base de datos	Pool de conexiones TCP reutilizado	Sin sockets TCP: driver HTTP de Neon
Variables de entorno	process.env	c.env, inyectado por peticion
Arranque en frio	Segundos si el proceso estaba dormido	13 ms medidos
Escalado	Manual o con autoscaling configurado	Automatico e implicito
Ubicacion	Una region	Cientos de ubicaciones, cerca del usuario

Punto debil que conviene reconocer

Sin pool de conexiones, **cada peticion abre una conexion HTTP nueva a Neon**. A la escala de esta prueba es irrelevante, pero con trafico alto seria el primer cuello de botella. La mitigacion natural es el endpoint *pooled* de Neon, que es precisamente el que se esta usando (el host lleva el sufijo *-pooler*): el pooling ocurre del lado de Neon, no del Worker.

4. Analisis y diseno de sistemas

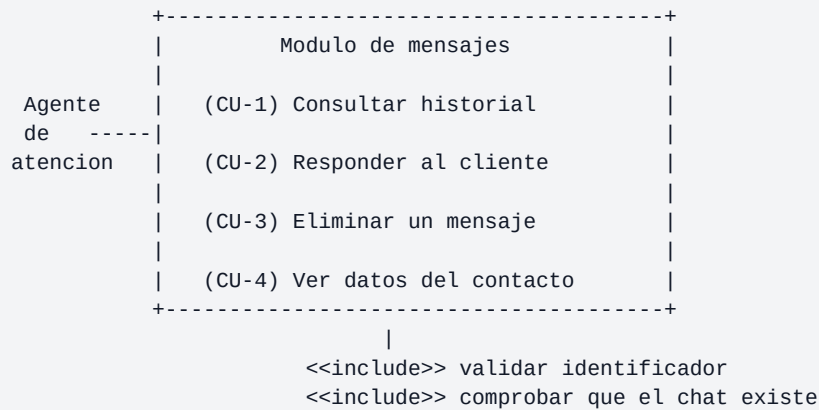
4.1 Requisitos funcionales

ID	Requisito	Como se verifica
RF-01	Listar los mensajes de un chat en orden cronologico ascendente	GET /chats/1/mensajes devuelve 8 mensajes ordenados; test unitario del formato
RF-02	Crear un mensaje de respuesta del negocio	POST con {contenido}; el registro se guarda con direccion = saliente
RF-03	La direccion la fija el servidor, nunca el cliente	Test dedicado que envia direccion=entrante y comprueba que se ignora
RF-04	Eliminar un mensaje	DELETE /mensajes/:id devuelve el registro borrado
RF-05	Rechazar identificadores no numericos con 400	Tres tests, uno por cada endpoint con parametro
RF-06	Rechazar contenido vacio o solo espacios con 400	Dos tests: cadena vacia y cadena de espacios
RF-07	Devolver 404 si el chat no existe	Tests en GET de chat, GET de mensajes y POST
RF-08	Interfaz de conversacion con distincion visual por direccion	Salientes a la derecha en azul, entrantes a la izquierda en gris
RF-09	Estados explicitos de carga, vacio y error	Cuatro pantallas distintas, incluida la de chatId invalido
RF-10	Validacion en cliente: no enviar mensajes vacios	El boton se deshabilita y el submit corta si el trim queda vacio

4.2 Requisitos no funcionales

Atributo de calidad	Decision de diseno que lo sostiene
Rendimiento	Ejecucion en el edge; validacion barata antes que consulta cara; count() en lugar de traer filas para contarlas
Escalabilidad	Sin estado en el servidor; escalado automatico de Workers; endpoint pooled de Neon
Seguridad	Secretos fuera del repositorio; consultas parametrizadas; CORS restrictivo; el cliente no decide la direccion
Mantenibilidad	Capas separadas; tipos inferidos del esquema; 15 tests que fijan el contrato
Usabilidad	Actualizaciones optimistas para respuesta inmediata; confirmacion antes de borrar; envio con Enter
Accesibilidad	role=log con aria-live en la lista, aria-label en los controles, foco visible, operable con teclado
Observabilidad	Health check independiente de la base de datos; logger de Hono; observabilidad activada en Workers
Portabilidad	Postgres estandar y Web APIs estandar: migrar a otro runtime edge no exigiria reescribir la logica

4.3 Casos de uso



CU-2 en detalle: responder al cliente

Elemento	Descripcion
Actor	Agente del equipo de atencion
Precondicion	El chat existe y esta abierto en el panel
Flujo principal	1. El agente escribe el mensaje. 2. Pulsa Enviar o la tecla Enter. 3. El mensaje aparece de inmediato en la conversacion (optimista). 4. El servidor lo persiste con direccion saliente. 5. La lista se revalida con los datos reales.
Flujo alternativo A	Contenido vacio o solo espacios: el boton esta deshabilitado y el envio no llega a salir
Flujo alternativo B	Fallo de red o del servidor: se revierte el mensaje optimista, se muestra el error y se restaura el texto escrito para no perderlo
Postcondicion	El mensaje queda persistido y visible para cualquier otro agente que abra el chat

4.4 Modelo entidad-relacion



Una empresa tiene N chats. Un chat tiene N mensajes.
 Cardinalidad 1:N en ambas relaciones, con integridad referencial
 garantizada por claves foraneas en la propia base de datos.

4.5 Diccionario de datos

Tabla	Columna	Tipo	Restricciones	Justificacion
empresas	id	SERIAL	PK	Identificador del tenant
empresas	nombre	TEXT	NOT NULL	Razon social
chats	id	SERIAL	PK	Identificador de la conversacion
chats	empresa_id	INTEGER	NOT NULL, FK	A que tenant pertenece
chats	nombre	TEXT	NOT NULL	Nombre del contacto, para la cabecera

Tabla	Columna	Tipo	Restricciones	Justificacion
chats	telefono	TEXT	NOT NULL	TEXT y no numerico: los telefonos llevan +, ceros a la izquierda y no se opera con ellos
mensajes	id	SERIAL	PK	Tambien sirve de desempate al ordenar
mensajes	chat_id	INTEGER	NOT NULL, FK	Conversacion a la que pertenece
mensajes	contenido	TEXT	NOT NULL	TEXT y no VARCHAR(n): en Postgres no hay ventaja de rendimiento y evita un limite arbitrario
mensajes	direccion	TEXT	NOT NULL	entrante o saliente
mensajes	created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Con zona horaria: los agentes y los clientes pueden estar en husos distintos

Defensa: por que TIMESTAMPTZ y no TIMESTAMP

TIMESTAMP guarda una fecha sin referencia horaria: si el servidor cambia de zona o el cliente esta en otro pais, el mismo instante se muestra distinto y no hay forma de saber cual es el correcto. TIMESTAMPTZ normaliza a UTC al guardar y convierte al leer. En un producto que atiende conversaciones de clientes finales en cualquier huso, es la unica opcion defendible.

5. Principios de programación aplicados

Este es el capítulo que más se pregunta y el que peor se suele responder. La clave no es recitar las siglas, sino **senalar la línea de código concreta** donde el principio se materializa. Cada apartado incluye esa evidencia.

5.1 Separación de responsabilidades (SoC)

Cada fichero tiene un motivo único para existir. La validación no sabe de HTTP, las rutas no construyen SQL, el acceso a datos no conoce los códigos de estado.

Fichero	Única responsabilidad
validations.ts	Definir que forma tiene una entrada válida
schema.ts	Definir que forma tienen los datos persistentes
routes/chat.ts	Orquestar el caso de uso y traducirlo a HTTP
db/index.ts	Fabricar la conexión
index.ts	Cablear infraestructura transversal

En el frontend la separación es análoga: `lib/api.ts` sabe hablar HTTP pero no conoce React; `lib/query.ts` conoce React Query pero no construye URLs; los componentes pintan pero no saben de donde vienen los datos.

5.2 Responsabilidad única (SRP)

Aplicado a funciones, no solo a ficheros. Ejemplo real del proyecto: el manejo del cuerpo de la petición se partió en dos piezas con motivos de cambio distintos.

```
// jsonBodyGuard: solo decide si el body es JSON parseable
const jsonBodyGuard = async (c, next) => {
  try { await c.req.json() }
  catch { return c.json({ status: "error", message: "El body debe ser JSON válido" }, 400) }
  await next()
}

// zValidator: solo decide si el contenido cumple las reglas de negocio
zValidator("json", createMensajeSchema, zodErrorHook)
```

Por qué existe jsonBodyGuard

Antes de añadirlo, un body malformado provocaba una excepción dentro del validador que acababa en el manejador global y devolvía **500**. Pero un JSON roto es culpa del cliente, no del servidor: corresponde un **400**. Esta corrección salió de probar la API con curl, no de leer el código. Es un buen ejemplo que contar: la diferencia entre 4xx y 5xx no es cosética, es semántica.

5.3 No te repitas (DRY)

Repetición evitada	Mecanismo
Formato de error en cada validación	<code>zodErrorHook</code> , un único manejador reutilizado por todos los validadores
Manejo de respuestas HTTP en el cliente	<code>handleResponse()</code> , un único punto que interpreta el contrato
Claves de cache de React Query	<code>apiKeys</code> , objeto centralizado en lugar de literales repartidos
Tipos de las entidades	Se infieren del esquema de Drizzle con <code>\$inferSelect</code> , no se escriben a mano
Combinación de clases CSS	<code>cn()</code> , envoltorio de <code>clsx</code> + <code>tailwind-merge</code>

DRY tiene un límite, y aquí se ve

Los tipos del frontend en `types/index.ts` **duplican** a mano la forma de las entidades del backend. Se podría haber extraído a un paquete compartido de un monorepo. Se optó por duplicar porque son dos despliegues independientes (Cloudflare y Vercel) y un paquete compartido habría obligado a montar workspaces y un pipeline de build común para un módulo de este tamaño. **Es deuda técnica consciente**, y decirlo así vale más que ocultarlo.

5.4 Fallar rapido (fail fast)

El sistema rechaza lo invalido en el punto mas temprano posible, antes de gastar recursos.

Orden real de las comprobaciones en POST /chats/:chatId/mensajes

- | | | |
|--------------------------|------------------------|--------|
| 1. Formato del parametro | regex, coste ~0 | -> 400 |
| 2. Body parseable | JSON.parse, coste bajo | -> 400 |
| 3. Reglas del contenido | Zod, coste bajo | -> 400 |
| 4. Existencia del chat | SELECT, coste de RED | -> 404 |
| 5. Insercion | INSERT, coste de RED | -> 201 |

Una peticion a /chats/abc/mensajes muere en el paso 1:
nunca abre una conexion a Neon.

5.5 Fuente unica de verdad

Dato	Donde se define una sola vez	Que se deriva de ahi
Forma de las tablas	db/schema.ts	Migraciones SQL, tipos Empresa/Chat/Mensaje, consultas tipadas
Forma de las entradas validas	utils/validations.ts	Validacion en ejecucion y tipos de TypeScript, via z.infer
URL de la API	NEXT_PUBLIC_API_URL	Todas las llamadas del cliente; nunca esta escrita en el codigo
Paleta y tipografia	tailwind.config.ts	Todas las clases de color de la interfaz

5.6 Nunca confiar en el cliente

El principio de seguridad mas visible del proyecto, y el que el enunciado pide de forma explicita. El cliente puede pedir lo que quiera; el servidor decide.

```
// El esquema de validacion NO incluye direccion
export const createMensajeSchema = z.object({
  contenido: z.string().trim().min(1).max(5000),
})

// Y el handler la fija literalmente
await db.insert(mensajes)
  .values({ chatId, contenido, direccion: "saliente" }) // <- no viene del body

// Zod descarta por defecto las claves no declaradas: si el cliente
// envia {"contenido":"x","direccion":"entrante"}, direccion se ignora.
```

Hay un test dedicado a esto que ademas comprueba, mediante un espia sobre la llamada de insercion, que el valor que llega a la base de datos es saliente. No basta con mirar la respuesta: se verifica lo que se persiste.

5.7 Idempotencia

Una operacion idempotente produce el mismo estado final se ejecute una vez o cien. El seed del proyecto lo es, y llegar ahi costo un error instructivo.

El error original y por que importa contarle

La primera version usaba `onConflictDoNothing()`, que **solo actua cuando existe una restriccion UNIQUE** sobre la que chocar. El esquema del enunciado no define ninguna, así que la clausula nunca se disparaba y cada ejecucion creaba una empresa y un chat nuevos. La solucion fue buscar antes de insertar. Se descarto anadir un UNIQUE para **no desviarse del modelo de datos que pedia el enunciado**: la idempotencia es responsabilidad del script de seed, no una excusa para cambiar el esquema.

5.8 El tipado como contrato ejecutable

TypeScript en modo strict no es decoracion: convierte en errores de compilacion cosas que de otro modo serian fallos en produccion. Los tipos de las entidades no se escriben, se infieren del esquema, de modo que renombrar una columna rompe la compilacion en todos los sitios que la usaban.

```
export type Mensaje = typeof mensajes.$inferSelect
// Derivado del esquema. Si mañana chat_id pasa a conversacion_id,
// el compilador senala cada uso en lugar de fallar en ejecucion.
```

6. Patrones de diseño identificables

No se aplicaron patrones por el gusto de aplicarlos. Estos son los que **están realmente en el código** y se pueden señalar con el dedo.

Patrón	Donde	Que resuelve
Cadena de responsabilidad (middleware)	index.ts y routes/chat.ts	Logging, CORS, conexión y validaciones se componen como eslabones. Cada uno puede cortar la cadena o delegar en el siguiente con next(). Anadir autenticación sería insertar un eslabón, sin tocar los handlers
Inyección de dependencias	c.set("db", getDb(c.env))	Los handlers reciben la base de datos en el contexto en lugar de importarla. Por eso los tests pueden sustituirla por un doble sin levantar Postgres
Factoría	getDb(env)	Encapsula como se construye la conexión. Necesario en el edge, donde no se puede crear un singleton al cargar el módulo porque las variables llegan por petición
DTO / contrato explícito	types/index.ts y el formato status/message	La frontera entre sistemas tiene una forma declarada y estable, independiente de como estén hechas las tablas por dentro
Guard clause	if (chat.length === 0) return 404	Se sale pronto en los casos excepcionales en lugar de anidar condicionales. El camino feliz queda al final, sin indentación
Optimistic UI	onMutate / onError / onSuccess	La interfaz se adelanta al servidor y revierte si falla. Convierte una espera de red en una respuesta aparentemente instantánea
Contenedor y presentación	page.tsx frente a ChatMessage y ChatInput	La página posee el estado y las mutaciones; los componentes reciben datos y callbacks. ChatMessage no sabe que existe React Query, así que se puede reutilizar o probar aislado
Adaptador	handleResponse()	Traduce el protocolo HTTP y el contrato de la API al mundo de excepciones de JavaScript, que es lo que React Query espera

Defensa: por que no hay capa de repositorio ni servicios

Una arquitectura hexagonal con repositorios, servicios de dominio y puertos sería **sobreingeniería** para tres endpoints CRUD. Drizzle ya funciona como capa de acceso a datos tipada. El criterio que se aplicó: *introducir una abstracción cuando exista un segundo caso que la justifique*. Si mañana hubiera que soportar otro motor de base de datos o reutilizar la lógica desde una cola de mensajes, ese sería el momento de extraer un repositorio. Saber cuando NO añadir una capa es tan importante como saber añadirla.

7. El contrato de la API

El enunciado impone un formato de respuesta concreto y lo llama *convencion real* de la empresa. Respetarlo al pie de la letra es parte de lo que se evalúa: demuestra que se leen las especificaciones antes de programar.

7.1 Formato de respuesta

```
// Exito
{ "status": "success", "mensajes": [ ... ], "total": 8, "limit": 50, "offset": 0 }
{ "status": "success", "mensaje": { ... } }
{ "status": "success", "chat": { ... } }

// Error
{ "status": "error", "message": "El contenido es requerido" }
```

El campo status aparece en **todas** las respuestas, incluidas las que genera la infraestructura: los errores de validación de Zod (a través de un hook propio), el 404 de ruta inexistente y el 500 del manejador global. No hay ninguna salida del sistema que se escape del contrato.

7.2 Endpoints

Metodo	Ruta	Entrada	Exito	Errores
GET	/health	-	200	-
GET	/chats/:chatId	param numerico	200 + chat	400, 404, 500
GET	/chats/:chatId/mensajes	param + limit/offset	200 + mensajes	400, 404, 500
POST	/chats/:chatId/mensajes	param + {contenido}	201 + mensaje	400, 404, 500
DELETE	/mensajes/:id	param numerico	200 + mensaje	400, 404, 500

Las rutas van **sin prefijo** /api, exactamente como las define el enunciado. Es una decisión deliberada: cuando una especificación fija rutas, cambiarlas por gusto propio rompe el contrato con quien la escribió.

7.3 Semántica de los códigos de estado

Código	Significado en este sistema	Ejemplo que lo dispara
200	Consulta o borrado correcto	GET /chats/1/mensajes
201	Recurso creado	POST que inserta un mensaje
400	La petición del cliente está mal formada	chatId no numérico, contenido vacío, JSON roto
404	La ruta o el recurso no existen	GET /chats/999 o una ruta desconocida
500	Fallo inesperado del servidor	Base de datos inalcanzable

Defensa: la frontera entre 400 y 500

Es una pregunta habitual. La regla que se sigue: **4xx significa que el cliente puede corregirlo; 5xx significa que no**. Por eso un JSON malformado devuelve 400 y no 500, aunque técnicamente sea una excepción lanzada dentro del servidor. Y por eso el 500 devuelve un mensaje genérico, sin traza de pila: el cliente no puede hacer nada con esa información y filtrarla ayudaría a un atacante.

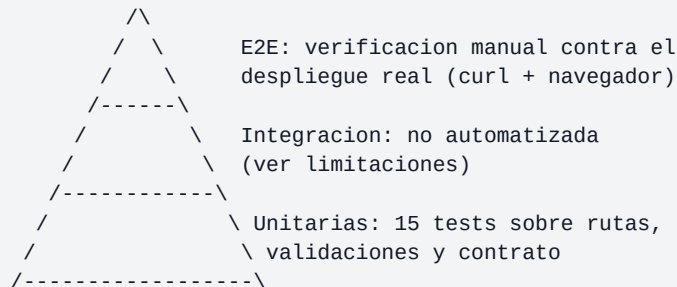
7.4 Reglas de validación

Entrada	Regla	Motivo
chatId, id	Debe casar con /^d+\$/	Rechaza negativos, decimales, notación científica y texto antes de tocar la base de datos
contenido	trim, mínimo 1	Una cadena de espacios no es un mensaje. El trim se aplica antes de medir
contenido	máximo 5000	Límite explícito, para no aceptar cargas arbitrariamente grandes
dirección	No se acepta del cliente	La fija el servidor
limit, offset	Números, por defecto 50 y 0	Evita paginaciones inválidas
body	Debe ser JSON parseable	Devuelve 400 en lugar de dejar escapar una excepción

8. Estrategia de pruebas

8.1 Que se prueba y por que ahí

La suite se concentra en la capa de rutas del backend, que es donde vive el contrato. La decision fue probar **comportamiento observable**, no detalles internos.



8.2 Los 15 tests

Bloque	Casos cubiertos
GET /chats/:chatId (3)	Devuelve nombre y telefono con formato success · 404 si no existe · 400 si el id no es numerico
GET /chats/:chatId/mensajes (3)	404 si el chat no existe · lista paginada con formato success y total correcto · 400 si el id no es numerico
POST /chats/:chatId/mensajes (6)	Crea con direccion saliente · ignora la direccion enviada por el cliente · rechaza contenido vacio · rechaza solo espacios · 400 (no 500) si el JSON es invalido · rechaza mas de 5000 caracteres
DELETE /mensajes/:id (3)	Elimina y devuelve el registro · 404 si no existe · 400 si el id no es numerico

8.3 Como se aísla la base de datos

Los tests sustituyen el modulo de base de datos por un doble, aprovechando que la conexion se inyecta en el contexto en lugar de importarse directamente. Sin esa inyeccion de dependencias, probar exigiria una base de datos real.

```
vi.mock("../src/db", () => ({ getDb: vi.fn(() => ({ select, insert, delete })) })))

// Cada test define que devuelve la consulta y comprueba la respuesta HTTP.
// El test de la direccion va mas alla y espia el valor que llega al insert:
expect(mockValues).toHaveBeenCalledWith(
  expect.objectContaining({ direccion: "saliente" })
)
```

8.4 Que NO cubren estos tests

Reconocerlo es parte de la defensa

Al usar dobles, los tests **no validan el SQL real ni el esquema de la base de datos**. Una columna mal escrita en una consulta pasaria los 15 tests y fallaria en produccion. Tampoco hay tests de interfaz. Como compensacion se hizo verificación manual sistemática: se inspecciono el SQL generado por Drizzle sin conectar (con `.toSQL()`), se probaron los cinco endpoints con curl contra Neon real, y se ejecuto el flujo completo de enviar y borrar en el despliegue de produccion desde el navegador. **Lo correcto a futuro son tests de integracion contra una base de datos efimera.**

9. Seguridad

9.1 Gestion de secretos

Entorno	Donde vive DATABASE_URL	Como llega al código
Scripts locales de base de datos	backend/.env (ignorado por git)	process.env, cargado por drizzle-kit y por --env-file
Worker en local	backend/.dev.vars (ignorado por git)	c.env, inyectado por wrangler dev
Produccion	Secret de Cloudflare	c.env, inyectado por el runtime

La distinción .env / .dev.vars merece explicación

Son dos ficheros con el mismo valor porque los leen dos mundos distintos. .env lo consumen procesos de Node (drizzle-kit, el script de seed) via process.env. .dev.vars lo lee wrangler para inyectarlo como *binding* del Worker, porque **en el edge no existe process.env**. Confundirlos hace que todos los endpoints devuelvan 500 mientras el health check sigue en verde: un síntoma muy característico que conviene saber diagnosticar.

9.2 CORS

La política se implementa con una **función**, no con una lista de cadenas. El motivo es concreto: Hono compara las cadenas de la lista de forma exacta, así que un comodín escrito como texto (por ejemplo `https://*.vercel.app`) **nunca casaría** y el frontend desplegado quedaría bloqueado.

```
origin: (origin) => {
  const isVercel = origin?.endsWith(".vercel.app") ?? false
  const isLocal = origin === "http://localhost:3000"
  return (isLocal || isVercel) ? origin : undefined
}

// Verificado: un Origin no permitido no recibe Access-Control-Allow-Origin.
```

9.3 Superficie de ataque y mitigaciones

Vector	Estado	Mecanismo
Inyección SQL	Mitigado	Drizzle emite consultas parametrizadas; los valores viajan como \$1, \$2, nunca concatenados
XSS almacenado	Mitigado	React escapa el contenido por defecto; no se usa dangerouslySetInnerHTML en ninguna parte
Manipulación de dirección	Mitigado	El servidor la fija; Zod descarta las claves no declaradas
Fuga de secretos al repositorio	Mitigado	.env y .dev.vars ignorados; historial verificado antes de publicar
CORS permisivo	Mitigado	Lista blanca por función, no comodín abierto
Fuga en mensajes de error	Mitigado	El 500 devuelve texto genérico, sin traza de pila
Autenticación y autorización	Ausente	Fuera de alcance del enunciado. Cualquiera puede leer y borrar mensajes de cualquier chat
Aislamiento multi-tenant	Ausente	La tabla empresas existe, pero ningún endpoint filtra por ella
Limitación de peticiones	Ausente	Solo la protección genérica de Cloudflare

Como defender lo que falta

La ausencia de autenticación no es un descuido, es el alcance del enunciado: pide tres endpoints sin mencionar sesiones ni usuarios. Lo importante es **saber exactamente que faltaria** para producción: un middleware de autenticación antes de las rutas, un identificador de empresa extraído del token, y un filtro por empresa_id en cada consulta. La arquitectura de middlewares en cadena está preparada para eso: sería añadir un eslabón, no reescribir los handlers.

10. Despliegue y operacion

10.1 Topologia

Componente	Proveedor	Como se despliega	Plan
API	Cloudflare Workers	wrangler deploy, manual	Gratuito
Frontend	Vercel	Automatico en cada push a master	Gratuito
Base de datos	Neon	Migraciones con drizzle-kit migrate	Gratuito
Codigo	GitHub	git push	Publico

10.2 Ciclo de migraciones

```
db:generate  Compara schema.ts con el estado anterior y escribe un .sql versionado
db:migrate   Aplica los .sql pendientes, sin interaccion <- el que se usa
db:push      Sincroniza directo comparando contra la base de datos (interactivo)
db:seed      Pueba datos de demostracion, de forma idempotente
```

Se prefiere migrate sobre push porque aplica ficheros versionados en git: el estado de la base de datos queda trazado en el historial y es reproducible. Ademas push es interactivo y pide confirmacion, lo que lo hace inservible en un pipeline automatizado.

10.3 La variable NEXT_PUBLIC_API_URL

Un detalle que se pregunta a menudo

El prefijo NEXT_PUBLIC_ hace que Next.js incruste el valor en el paquete de JavaScript **en tiempo de compilacion**, no de ejecucion. Consecuencia practica: cambiar la variable en el panel de Vercel **no surte efecto hasta que se vuelve a desplegar**. Es una fuente clasica de confusion. Tambien implica que ese valor es publico y visible en el navegador, motivo por el que ahi solo puede ir una URL, jamas un secreto.

10.4 Incidencias reales del despliegue

Merece la pena tener preparadas estas cuatro: demuestran que el sistema se llevo hasta produccion de verdad y no solo hasta que compilaba.

Sintoma	Causa	Solucion
El deploy falla con <i>CPU limits are not supported for the Free plan</i>	wrangler.toml declaraba un bloque [limits] que solo admite el plan de pago	Eliminar el bloque; el limite del plan gratuito ya es el que se pretendia fijar
db:seed responde <i>DATABASE_URL no configurada</i> aunque el .env es correcto	tsx no carga ficheros .env por si mismo, a diferencia de drizzle-kit	Ejecutarlo con --env-file-if-exists=.env
El Worker sube pero no tiene URL publica	La cuenta no tenia registrado un subdominio workers.dev	Registrarlo una vez; es permanente por cuenta
En local todo devuelve 500 excepto /health	Falta .dev.vars: wrangler no lee .env para los bindings del Worker	Crear .dev.vars con el mismo valor. Que /health siga en verde es la pista: es el unico endpoint que no toca la base de datos

11. Defensa: preguntas y respuestas

Treinta preguntas ordenadas de mas probable a menos. La respuesta esta redactada para decirse en voz alta en menos de un minuto.

P01. Explicame la arquitectura en un minuto.

Tres piezas desacopladas. Un frontend Next.js en Vercel que solo pinta y gestiona estado de servidor con React Query. Una API Hono desplegada en el edge de Cloudflare, organizada en cuatro capas: entrada, validacion, rutas y acceso a datos. Y Postgres serverless en Neon, con Drizzle como capa tipada. El navegador llama directamente a la API, por eso hay CORS.

P02. Por que Hono y no Express?

Express depende de las APIs HTTP de Node, que no existen en un runtime edge. Hono trabaja sobre Request y Response de la Web Fetch API, que es justo lo que expone Workers. Ademas pesa unos 14 KB, lo cual importa cuando el bundle se despliega globalmente.

P03. Como garantizas que un mensaje creado por la API sea siempre saliente?

De dos formas que se refuerzan. El esquema de validacion no declara el campo direccion, y Zod descarta por defecto las claves que no declara. Y el handler lo escribe literalmente en el insert. Hay un test que envia direccion entrante y comprueba, espiando la llamada al insert, que lo que llega a la base de datos es saliente.

P04. Que pasa si el cliente manda un JSON roto?

Devuelve 400 con el contrato de error. Al principio devolvía 500, porque la excepcion del parseo escapaba al manejador global. Lo detecte probando con curl y anadi un middleware que intenta parsear primero. Un JSON malformado es error del cliente, no del servidor.

P05. Por que TIMESTAMPTZ?

Porque guarda el instante con referencia horaria. Con TIMESTAMP, si el servidor cambia de zona o el cliente esta en otro pais, el mismo momento se muestra distinto y no hay forma de saber cual es el correcto. En un producto que atiende conversaciones en cualquier huso, no hay alternativa.

P06. Como ordenas los mensajes y por que anadiste el id?

ORDER BY created_at ascendente, con el id como desempate. Lo anadi despues de ver que los ocho mensajes del seed caian dentro del mismo segundo contra la base de datos real: cuando dos timestamps coinciden, sin desempate el orden lo decide el planificador y la conversacion puede aparecer barajada.

P07. Que son las actualizaciones optimistas y como manejas el fallo?

La interfaz asume que la operacion va a salir bien y pinta el resultado antes de que responda el servidor. En onMutate guardo una instantanea de la cache y aplico el cambio; si falla, onError la restaura; y en onSuccess invalido para traer los datos reales. Ademas, si el envio falla se restaura el texto escrito para que el usuario no lo pierda.

P08. Por que React Query y no useState con useEffect?

Porque los mensajes son estado de servidor, no de cliente. Con useEffect tendria que escribir a mano cache, deduplicacion, reintentos, estados de carga y error, invalidacion y rollback optimista. React Query es exactamente esa maquinaria, ya probada.

P09. Como aislas la base de datos en los tests?

La conexion se inyecta en el contexto de Hono mediante un middleware, no se importa directamente en los handlers. Eso permite sustituir el modulo por un doble en los tests. Es inyeccion de dependencias, y es lo que hace la suite rapida y sin infraestructura.

P10. Que NO cubren tus tests?

Al usar dobles, no validan el SQL real ni el esquema. Una columna mal escrita pasaria los 15 tests y fallaria en produccion. Tampoco hay tests de interfaz. Lo compense con verificacion manual: inspeccione el SQL generado con `.toSQL()`, probe los cinco endpoints con curl contra Neon y ejecute el flujo completo en el despliegue real. Lo correcto a futuro son tests de integracion contra una base de datos efimera.

P11. Por que las rutas no llevan prefijo /api?

Porque el enunciado las especifica sin prefijo. Cuando una especificacion fija el contrato, cambiarlo por preferencia personal lo rompe. Si esta API conviviera con otras en el mismo dominio, el prefijo tendria sentido; aqui es un Worker dedicado.

P12. Por que el health check no consulta la base de datos?

Deliberado. Esta montado antes del middleware que inyecta la conexion. Asi, si Neon esta caido, `/health` sigue respondiendo 200 y me dice que el Worker vive y que el problema esta en la base de datos. Si dependiera de ella, ambos fallos serian indistinguibles.

P13. Como funciona tu politica de CORS?

Con una funcion que evalua el Origin, no con una lista de cadenas. Es importante porque Hono compara la lista de forma exacta: un comodin escrito como texto nunca casaria y el frontend desplegado quedaria bloqueado. La funcion acepta `localhost:3000` y cualquier subdominio de `vercel.app`.

P14. Como manejas los errores inesperados?

Con `app.onError`, un manejador global que captura cualquier excepcion y devuelve 500 con el contrato, sin traza de pila. Es equivalente a poner `try/catch` en cada handler pero sin repetirlo cinco veces. Si tuviera que justificarlo ante quien pidio literalmente `try/catch`: cumple la misma funcion, el servidor no se cae y la respuesta respeta el formato.

P15. Que es un tenant y donde esta el multi-tenancy aqui?

Cada empresa cliente es un tenant. La tabla empresas es la raiz de esa separacion y los chats cuelgan de ella. Ahora bien, siendo honesto: **el aislamiento no esta implementado**. Ningun endpoint filtra por `empresa_id` porque no hay autenticacion. El modelo esta preparado; el control de acceso, no.

P16. Por que Drizzle y no Prisma?

Prisma arrastra un motor binario y un cliente generado que no funcionan en un runtime edge sin su Data Proxy de pago. Drizzle es una capa fina sobre el driver, se empaqueta sin problema, genera SQL legible y deriva los tipos del esquema.

P17. Como evitas la inyeccion SQL?

Drizzle emite siempre consultas parametrizadas: los valores viajan como `$1`, `$2` y nunca se concatenan en la cadena SQL. Lo verifique imprimiendo el SQL generado con `.toSQL()`. Ademas los identificadores se validan con una expresion regular antes de llegar a la consulta.

P18. Y el XSS?

React escapa el contenido por defecto al renderizar. No uso `dangerouslySetInnerHTML` en ninguna parte, que es el único camino habitual para introducir XSS en React.

P19. Que principios de programación aplicaste?

Separación de responsabilidades por capas; responsabilidad única hasta el nivel de función; DRY con un único manejador de errores de validación y una única función que interpreta las respuestas; fail fast ordenando las comprobaciones de barata a cara; fuente única de verdad, con los tipos inferidos del esquema; y no confiar en el cliente.

P20. Que patrones de diseño hay en el código?

Cadena de responsabilidad en los middlewares; inyección de dependencias con la conexión en el contexto; factoría en `getDb`; guard clauses en los handlers; optimistic UI en las mutaciones; y contenedor/presentación entre la página y los componentes.

P21. Por qué no hay capa de repositorio ni servicios?

Sería sobreingeniería para tres endpoints CRUD. Drizzle ya actúa como capa de acceso tipada. Mi criterio es introducir una abstracción cuando exista un segundo caso que la justifique. Si hubiera que soportar otro motor o reutilizar la lógica desde una cola, extraería el repositorio entonces.

P22. Cual es el cuello de botella si esto crece?

La base de datos. En el edge no hay pool de conexiones, así que cada petición abre una conexión HTTP a Neon. Por eso uso el endpoint `pooled`, que hace el pooling del lado de Neon. El siguiente paso sería cachear la lectura del chat, que cambia poco, y paginar de verdad con cursores.

P23. Por qué paginación con limit y offset y no con cursor?

Simplicidad, y suficiente para el volumen de un chat. Offset degrada en tablas grandes porque la base de datos tiene que recorrer y descartar las filas saltadas. Con historiales largos usaría paginación por cursor sobre `created_at` más `id`, que es justamente el índice que ya necesito para ordenar.

P24. Que es idempotencia y donde la aplicaste?

Que ejecutar una operación una vez o cien deje el mismo estado. Lo apliqué en el seed. Mi primera versión usaba `onConflictDoNothing`, que solo actúa si existe una restricción `UNIQUE`, y el esquema del enunciado no define ninguna: nunca se disparaba y duplicaba datos. Lo cambie a buscar antes de insertar, sin añadir `UNIQUE` para no desviarme del modelo pedido.

P25. Por qué el frontend llama directo a Workers en vez de pasar por Next.js?

Porque añadir un proxy metería un salto de red y una función serverless más en cada petición sin ganar nada: la API es pública y no hay secretos que ocultar en ese trayecto. El precio es configurar CORS, que es justo lo que pide el enunciado.

P26. Que diferencia hay entre .env y .dev.vars?

Los leen dos mundos distintos. `.env` lo consumen procesos de Node vía `process.env`, como `drizzle-kit` y el seed. `.dev.vars` lo lee `wrangler` para inyectarlo como binding del Worker, porque en el edge no existe `process.env`. Si confundes los dos, todos los endpoints devuelven 500 y solo `/health` sigue en verde.

P27. Como se comporta la interfaz si la API esta caida?

Muestra una pantalla de error con boton de reintentar. Reconozco un fallo de detalle: el mensaje que ve el usuario puede ser el crudo del navegador, Failed to fetch, en lugar de algo legible. Mapear los errores de red a un texto entendible es una mejora pendiente identificada.

P28. Que accesibilidad tiene la interfaz?

La lista es un role=log con aria-live polite, de modo que un lector de pantalla anuncia los mensajes nuevos. Los controles tienen aria-label, el foco es visible, se envia con Enter y salto de linea con Shift+Enter, y el boton de borrar es alcanzable con teclado porque responde tambien a focus, no solo a hover.

P29. Que harias distinto si tuvieras una semana mas?

Por este orden: autenticacion con aislamiento por empresa, tests de integracion contra una base de datos efimera, tiempo real con WebSockets o SSE en lugar de invalidar la cache, paginacion por cursor con scroll infinito, y tipos compartidos en un paquete comun en lugar de duplicados.

P30. Cual es la parte de la que estas mas orgulloso?

De haberlo llevado a produccion de verdad y haber encontrado cuatro fallos que solo aparecen ahi: el bloque de limites que el plan gratuito rechaza, el seed que no cargaba el .env, el subdominio sin registrar y los 500 por confundir .env con .dev.vars. Compilar no es funcionar.

12. Limitaciones conocidas y hoja de ruta

Enumerarlas antes de que las encuentre el evaluador cambia el tono de la conversacion: pasa de auditoria a discusion tecnica entre iguales.

12.1 Deuda tecnica reconocida

Limitacion	Impacto	Por que se acepto
Sin autenticacion ni autorizacion	Alto en produccion, nulo en la prueba	Fuera del alcance del enunciado, que pide tres endpoints sin mencionar usuarios
Aislamiento multi-tenant no aplicado	Alto en produccion	Depende de tener autenticacion primero: sin identidad no hay a que filtrar
Tests con dobles, sin integracion real	Medio	Exigiria levantar una base de datos efimera en el pipeline; se compenso con verificacion manual documentada
Tipos duplicados entre back y front	Bajo	Un paquete compartido obligaria a montar un monorepo con build comun para un modulo pequeno
Los errores al borrar no se muestran	Bajo	Detectado en la auditoria propia. Si el borrado falla, el mensaje reaparece sin explicacion
Errores de red sin traducir	Bajo	El usuario puede ver Failed to fetch en lugar de un texto entendible
Paginacion por offset	Bajo al volumen actual	Degrada en historiales muy largos; el cambio a cursor esta identificado
Sin limitacion de peticiones propia	Bajo	Solo la proteccion generica de Cloudflare por delante

12.2 Hoja de ruta ordenada por valor

Prioridad	Mejora	Justificacion
1	Autenticacion y filtrado por empresa_id	Sin ello el sistema no es publicable. Encaja como un middleware mas en la cadena existente
2	Tests de integracion contra base de datos efimera	Es el hueco real de la suite actual: cerraria la brecha entre SQL probado y SQL ejecutado
3	Tiempo real con WebSockets o SSE	Hoy la lista se refresca al invalidar la cache. Un chat deberia recibir mensajes entrantes sin recargar
4	Paginacion por cursor con scroll infinito	Necesario en conversaciones largas; el indice created_at + id ya esta implicito en el orden
5	Tipos compartidos en un paquete comun	Eliminarla la duplicacion manual y haria imposible que back y front se desincronicen
6	Traduccion de errores de red y aviso al fallar el borrado	Detalles de experiencia de usuario ya identificados en la auditoria

13. Glosario

Terminos que conviene saber definir con una frase propia si preguntan.

Termino	Definicion
Edge computing	Ejecutar código en servidores repartidos geográficamente, cerca del usuario, en lugar de en una única región.
Serverless	Modelo en el que no se gestiona ni escala ningún servidor: el proveedor asigna recursos por invocación.
Aislado de V8	Contexto de ejecución ligero dentro del motor de JavaScript. Arranca en milisegundos, frente a los cientos de un contenedor.
Arranque en frío	Latencia extra de la primera petición cuando el entorno de ejecución no estaba activo.
Multi-tenant	Una sola instancia de software sirviendo a múltiples clientes con sus datos separados lógicamente.
ORM	Capa que traduce entre objetos del lenguaje y filas de una base de datos relacional.
Migración	Fichero versionado que describe un cambio del esquema, de modo que el estado de la base de datos sea reproducible.
Idempotencia	Propiedad de una operación que produce el mismo estado final se ejecute una o muchas veces.
CORS	Mecanismo del navegador que decide si una página de un origen puede leer respuestas de otro origen.
Estado de servidor	Datos que viven en el backend y de los que el cliente solo tiene una copia cacheada, posiblemente obsoleta.
Actualización optimista	Pintar el resultado esperado antes de que responda el servidor, revirtiendo si falla.
Server Component	Componente de React que se renderiza en el servidor y cuyo código no viaja al navegador.
Hidratación	Proceso por el que React adjunta interactividad al HTML que llegó ya renderizado.
DTO	Objeto cuya única función es transportar datos entre capas o sistemas, con una forma declarada.
Guard clause	Retorno temprano ante un caso excepcional, para evitar anidar condicionales.
Consulta parametrizada	Consulta en la que los valores viajan separados del texto SQL, lo que impide la inyección.
Fail fast	Rechazar lo inválido en el punto más temprano posible, antes de consumir recursos.
Binding	Recurso o variable que el runtime de Workers inyecta en el entorno de ejecución de la petición.

Último consejo para la exposición

Cuando no sepas algo, la respuesta correcta es *no lo sé, pero lo comprobaría así* y describir el método. Un evaluador técnico valora mucho más eso que una respuesta inventada, porque lo que está midiendo es como piensas, no cuanto memorizas.

A. Apendice: auditoria adversarial

Antes de la exposición se sometió el sistema desplegado a dos auditorías independientes: una de **ataque a la API** (59 peticiones contra producción) y otra de **revisión documental bloque por bloque**. Los resultados están aquí porque conocerlos de antemano convierte una posible emboscada en una

Un `\u0000` es una cadena válida en JavaScript pero Postgres no lo admite en columnas TEXT. La validación debería rechazarlo antes del insert. El mensaje no llega a persistirse y no hay fuga, pero el código de estado es el equivocado.

A.2.4 Ausencia de índices (severidad baja hoy, alta a escala)

No hay ningún índice más allá de las claves primarias. Con un chat de cien mil mensajes, `WHERE chat_id = ? ORDER BY created_at, id` degenera en un recorrido secuencial, y el `count()` recorre todas las filas del chat en cada página. La corrección es un índice compuesto sobre `(chat_id, created_at, id)`, que además es exactamente el que necesitaría la paginación por cursor.

A.3 Imprecisiones de documentacion detectadas

Todas ellas son munición fácil para un evaluador, porque se comprueban leyendo dos ficheros del propio repositorio y viendo que se contradicen.

Afirmacion	Realidad
El README raíz documenta CHECK (direccion IN (...))	No existe tal restricción. La garantía es de aplicación, no de base de datos. El README del backend lo documenta bien: los dos documentos se contradicen
Se describe el sistema como multi-tenant	El modelo lo soporta, pero ninguna ruta filtra por empresa_id y no hay autenticación. Es la afirmación más fácil de tumbar con una sola pregunta
<code>npm test -- --coverage</code>	Falla: falta el paquete de cobertura en las dependencias
<code>npm run lint</code>	No funciona: no hay fichero de configuración de ESLint
La URL de la API nunca está hardcodeada	Hay un fallback a localhost:8787. Si falta la variable en Vercel, el build no falla y se despliega un frontend roto apuntando a localhost
Comentario en query.ts: <i>refetch en foco de ventana</i>	Nueve líneas más abajo el código pone <code>refetchOnWindowFocus: false</code>
Comentario en validations.ts: <i>previenen inyección SQL</i>	Zod valida forma; quien previene la inyección es Drizzle al parametrizar
Comentario en db/index.ts: <i>se llama dentro de cada handler</i>	Se llama una sola vez en un middleware global. La implementación real es mejor que la descrita

La lección de fondo

Ninguno de estos hallazgos compromete datos ni tumba el sistema. Lo que revelan es algo más sutil y más útil: **la documentación y los comentarios envejecen más rápido que el código**. Un comentario que miente es peor que no tener comentario, porque induce a error a quien lo lee. Si en la exposición surge alguno de estos puntos, la mejor respuesta es admitirlo, explicar cómo se detectó (auditoría sistemática previa) y decir cuál es la corrección.

A.4 Preguntas incómodas y cómo encajarlas

Dices multi-tenant, pero cualquiera puede leer el chat de cualquier empresa.

Cierto, y es un error de redacción mío. El modelo de datos está preparado para multi-tenant, con `empresa_id` en chats, pero el aislamiento no está implementado porque no hay autenticación, que queda fuera del alcance del enunciado. El siguiente paso concreto sería un middleware de autenticación y un filtro por `empresa_id` en las cuatro consultas.

Tu README documenta un CHECK que no existe en la base de datos.

Tiene razón. Es una divergencia entre el README raíz y el esquema real; el README del backend sí lo documenta correctamente. La restricción de valores vive hoy en la aplicación, no en la base de datos. Añadir el CHECK sería una mejora barata y sensata.

Que pasa con un chat de cien mil mensajes?

Se degrada, y se exactamente por que: no hay índices más allá de las claves primarias, el count recorre todas las filas en cada página y el offset es lineal. La corrección es un índice compuesto sobre `chat_id`, `created_at`, `id`, y cambiar a paginación por cursor, que aprovecharía ese mismo índice.

Tu CORS acepta cualquier subdominio de vercel.app.

Correcto. Lo elegí para que funcionaran los despliegues de previsualización, cuyo subdominio cambia en cada commit. Hoy el riesgo es nulo porque no hay sesiones ni cookies, pero en producción lo endurecería a una lista blanca exacta por variable de entorno y quitaría credentials.

Por que ?limit no tiene tope?

Es un descuido real. Válido que sea numérico pero no acote. Un limit muy alto intentaría traerlo todo. Debería llevar un máximo, del orden de cien.